SSO Microservices Architecture

Cloud Computing Lab Project — Docker Hub Deployment

1. Project Links

GitHub Repository

URL: https://github.com/satyam7535/sso-project

Docker Hub Repositories

All microservice images are publicly hosted on Docker Hub:

Service	Docker Hub URL	Pull Command
Auth Service	https://hub.docker.com/r/kesharisatyam/sso-auth-service	`docker pull kesharisatyam/sso-auth-service`
App 1 (Login & OAuth)	https://hub.docker.com/r/kesharisatyam/sso-app1	`docker pull kesharisatyam/sso-app1`
App 2 (Protected App)	https://hub.docker.com/r/kesharisatyam/sso-app2	`docker pull kesharisatyam/sso-app2`

Docker Hub Screenshot

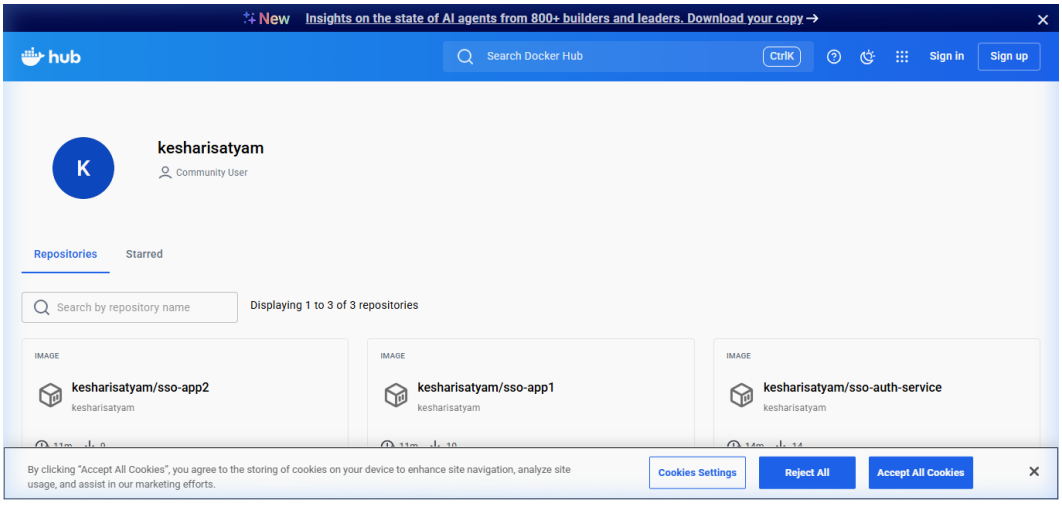

Figure: Docker Hub profile showing all 3 repositories

GitHub Repository Screenshot

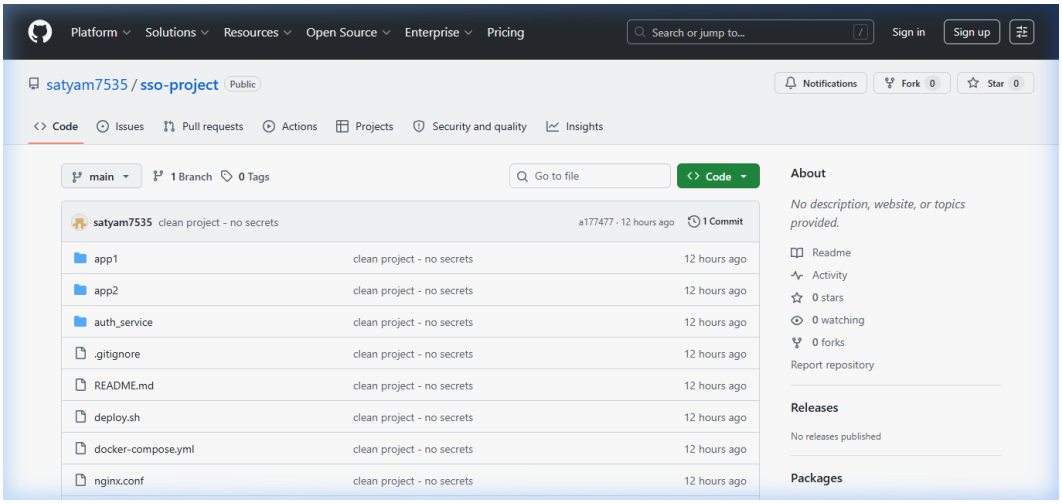

Figure: GitHub repository — satyam7535/sso-project

2. Environment Variables (.env File)

The project requires a .env file in the root directory with the following Google OAuth 2.0 credentials for social login functionality:

```
# .env file
GOOGLE_CLIENT_ID=795307984050-
9b0uecp675vc44n6c0qj1olqksrs7irm.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=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
```

.env File Screenshot

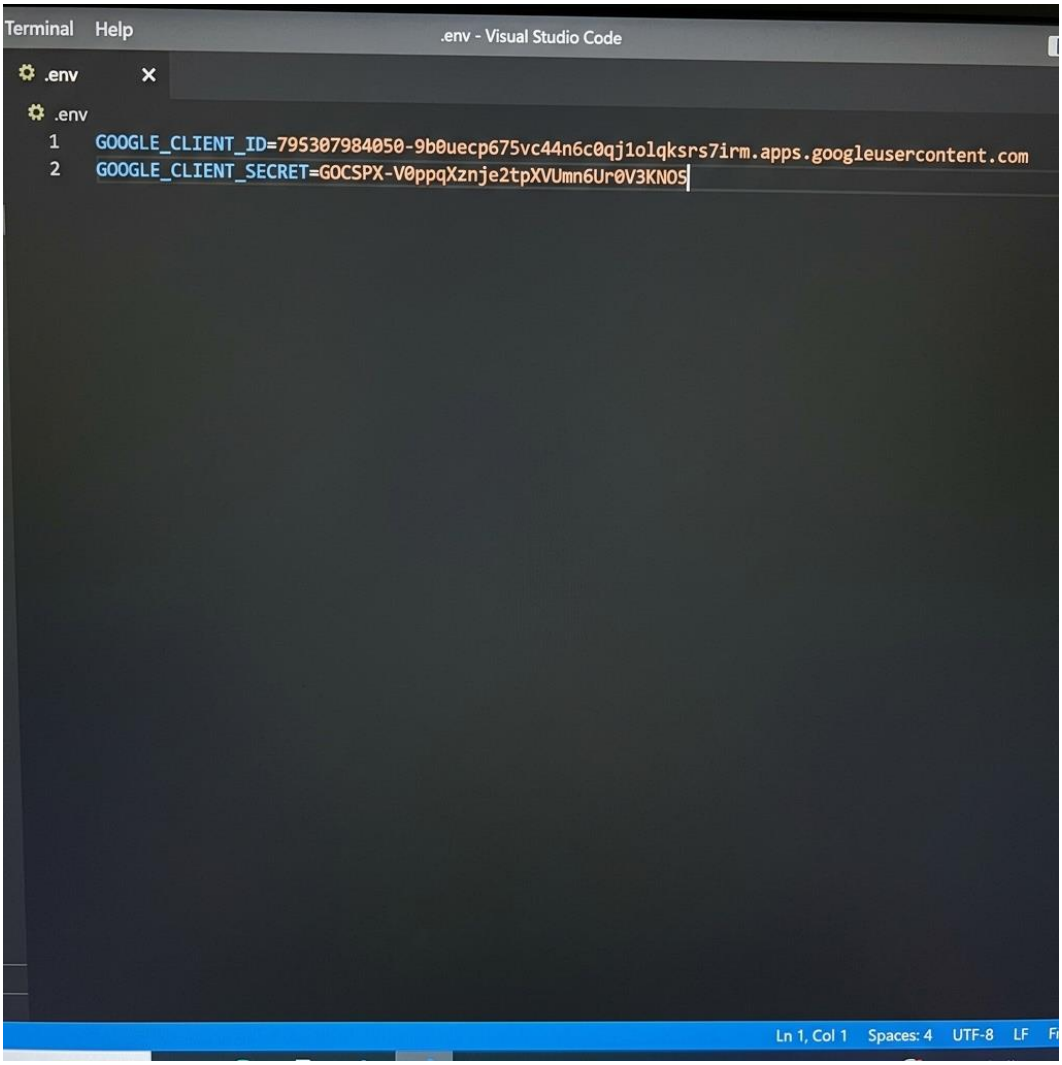

Figure: .env file containing Google OAuth credentials

3. Prerequisites (What You Need)

Requirement	Purpose	Download Link
Docker Desktop	Runs containers on your machine	https://www.docker.com/products/docker-desktop
Docker Compose	Orchestrates multi-container apps (included with Docker Desktop)	Included with Docker Desktop
Git (Optional)	Clone the project source code	https://git-scm.com/downloads
.env File	Google OAuth	Create manually (see Section 2)

	credentials for social login	
Internet Connection	Pull images from Docker Hub	N/A

4. How to Run the Project

Step 1: Install Docker Desktop

Download and install Docker Desktop from https://www.docker.com/products/docker-desktop

Make sure Docker Desktop is running before proceeding.

Step 2: Create docker-compose.yml

Create a new folder and save the following as docker-compose.yml:

```
services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: sso_user
      POSTGRES_PASSWORD: sso_password
      POSTGRES_DB: sso_db
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  auth-service:
    image: kesharisatyam/sso-auth-service:latest
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgres://sso_user:sso_password@db:5432/sso_db
      - DEBUG=False
      - ALLOWED_HOSTS=*
      - TOKEN_EXPIRY_MINUTES=30
      - INTERNAL_SSO_KEY=your-internal-sso-key-here
    depends_on:
      - db

  app1:
    image: kesharisatyam/sso-app1:latest
    ports:
      - "8001:8001"
    environment:
      - DATABASE_URL=postgres://sso_user:sso_password@db:5432/sso_db
      - DEBUG=True
      - ALLOWED_HOSTS=*
      - AUTH_SERVICE_URL=http://auth-service:8000
      - APP2_URL=http://127.0.0.1:8002
      - GOOGLE_CLIENT_ID=<your-google-client-id>
      - GOOGLE_CLIENT_SECRET=<your-google-client-secret>
      - SSO_COOKIE_DOMAIN=localhost
      - INTERNAL_SSO_KEY=your-internal-sso-key-here
      - SITE_DOMAIN=127.0.0.1:8001
    depends_on:
      - auth-service

  app2:
    image: kesharisatyam/sso-app2:latest
```

```
    ports:
      - "8002:8002"
    environment:
      - DATABASE_URL=postgres://sso_user:sso_password@db:5432/sso_db
      - DEBUG=False
      - ALLOWED_HOSTS=*
      - AUTH_SERVICE_URL=http://auth-service:8000
      - APP1_LOGIN_URL=http://127.0.0.1:8001/login/
    depends_on:
      - auth-service

volumes:
  postgres_data:
```

Step 3: Run the Project

Open a terminal/PowerShell in the folder containing docker-compose.yml and run:

```
docker-compose up
```

Docker will automatically pull all images from Docker Hub and start the services.

Step 4: Access the Application

Service	URL	Description
App 1 (Login)	http://127.0.0.1:8001	Main login page with Google OAuth
App 2 (Dashboard)	http://127.0.0.1:8002	Protected app — requires SSO login
Auth Service (API)	http://127.0.0.1:8000	Token validation API (backend)

5. Technology Stack

Technology	Purpose
Python / Django	Backend web framework for all 3 services
PostgreSQL	Relational database for user & token storage
Docker & Docker Compose	Containerization and orchestration
Google OAuth 2.0	Social authentication (Sign in with Google)
Gunicorn	Production-grade WSGI HTTP server
REST APIs	Inter-service communication for token validation
Nginx (optional)	Reverse proxy for production deployment

6. Architecture Overview

The system follows a microservices architecture with three independent Django services:

1. Auth Service (Port 8000) — Central authentication server that manages user accounts and SSO tokens. All token creation and validation goes through this service.
2. App 1 (Port 8001) — The login portal with both traditional username/password and Google OAuth. After successful authentication, it obtains a token from Auth Service and sets it as a cookie.
3. App 2 (Port 8002) — A protected application that validates the SSO cookie by calling Auth Service. If the token is valid, the user gets access; otherwise, they are redirected to App 1 for login.

Database: All services share a single PostgreSQL database for user and token data.